# Q2. Goal-Based News Agent with a Genetic Algorithm

> **Question.** For a "news reporting" product, design a goal-based agent that uses a genetic algorithm (GA) in its agent function. S is the original set of news and G is the target news with the expected reader impact. Every square in Diagram 2 has a fitness and comes from legal moves with genetic operators. Give the detailed design: an initial population of at least four news items, four genetic operators, a fitness function, pseudocode, the working algorithm and a discussion of exit criteria. No more than two black squares may be visited in a row.

## 1. The idea in plain words

Think of Diagram 2 as a **map of every version of one news story**. S is the story as first written. G is the version that gives the impact we want on readers. Each square is a version, and a move to the next square is one edit: swap the headline, add a fact, change the tone, cut a sensational line. The GA does not walk one path. It grows a whole *population* of edit sequences, keeps the ones that get closest to G, mixes them and mutates them. After some generations, one sequence reaches G safely.

## 2. Assumptions

- The map has 7 × 12 squares. **Grey** squares are acceptable versions (quality 0.7). **Black** squares are risky versions: sensational, unverified or biased (quality 0.2).
- **C and D** are editorial checkpoints (fact-check desk and legal desk). Passing through them earns a small bonus.
- Rule from the question: **no more than two black squares in a row.** A third black square in a row is a violation.
- An edit that would leave the map is skipped (that operator does nothing).

## 3. Agent design

| Part | News agent |
|---|---|
| Agent type | **Goal-based:** it knows the goal G and plans edits to reach it. The GA is its planning step. |
| Performance measure | Gap to target impact, accuracy and quality of the story, rule obeyed, few edits |
| Environment | The story versions, readers, editors, fact sources |
| Sensors | News feeds, reader clicks and shares, fact-check results |
| Actuators | The four edit operators, then publish or send to an editor |
| Agent function | Run the GA on the percept (current story S), then publish the best sequence's final version |

### Initial population (four news items)

Each news item is a chromosome with seven genes: headline, lead, facts, sources, tone, length and image.

| Item | Headline | Facts | Sources | Tone | Length | Risk |
|---|---|---|---|---|---|---|
| N1 Bridge | "SHOCK: city bridge collapses!" | 2 | 1 | alarming | short | black |
| N2 Health | "New flu strain: what we know" | 5 | 3 | calm | medium | grey |
| N3 Tech | "Phone maker unveils foldable" | 4 | 2 | neutral | medium | grey |
| N4 Sports | "UNBELIEVABLE final-over win" | 3 | 1 | excited | short | black |

In the run, the GA's own population is 60 **operator sequences** (30 edits each) applied to the story, which is what moves a version across the map.

### The four genetic operators

| Move | Operator | What it does to the story |
|---|---|---|
| L | Crossover | Take the headline of one news item and the body of another |
| D | Fact insertion (mutation) | Add a verified fact, quote or source |
| R | Tone mutation | Rewrite the tone, e.g. alarming → calm |
| U | Deletion / inversion | Remove a sensational line, or reorder into an inverted pyramid |

Real example from the run, on "SHOCK: city bridge collapses, thousands trapped!": the delete operator gives "City bridge collapses, thousands trapped", and the tone mutation gives "Bridge section collapses in city; rescue under way".

### Fitness function

**F = 0.45·closeness + 0.30·reached + 0.15·avg quality + 0.05·checkpoints − 0.12·violations − 0.002·length**

- **closeness** = 1 − (distance from the final square to G) / 17. This is the gap to the target impact.
- **reached** = 1 if the path ends on G. **avg quality** = mean quality of the visited squares.
- **checkpoints** = how many of C and D the path passes (0–2).
- **violations** = number of squares that make a run of three or more black squares. This is the penalty for the rule.

## 4. Working algorithm (step by step)

1. Create 60 random operator sequences. Each starts at S.
2. Run each sequence on the map and compute its fitness.
3. Check the exit criteria (section 6). If met, return the best path.
4. Pick parents by tournament (best of 3 random sequences).
5. Crossover: join the first part of one parent with the second part of the other.
6. Mutate: each gene has a 4% chance of becoming a random operator.
7. Copy the 2 best sequences unchanged (elitism), fill the rest with children and go to step 2.

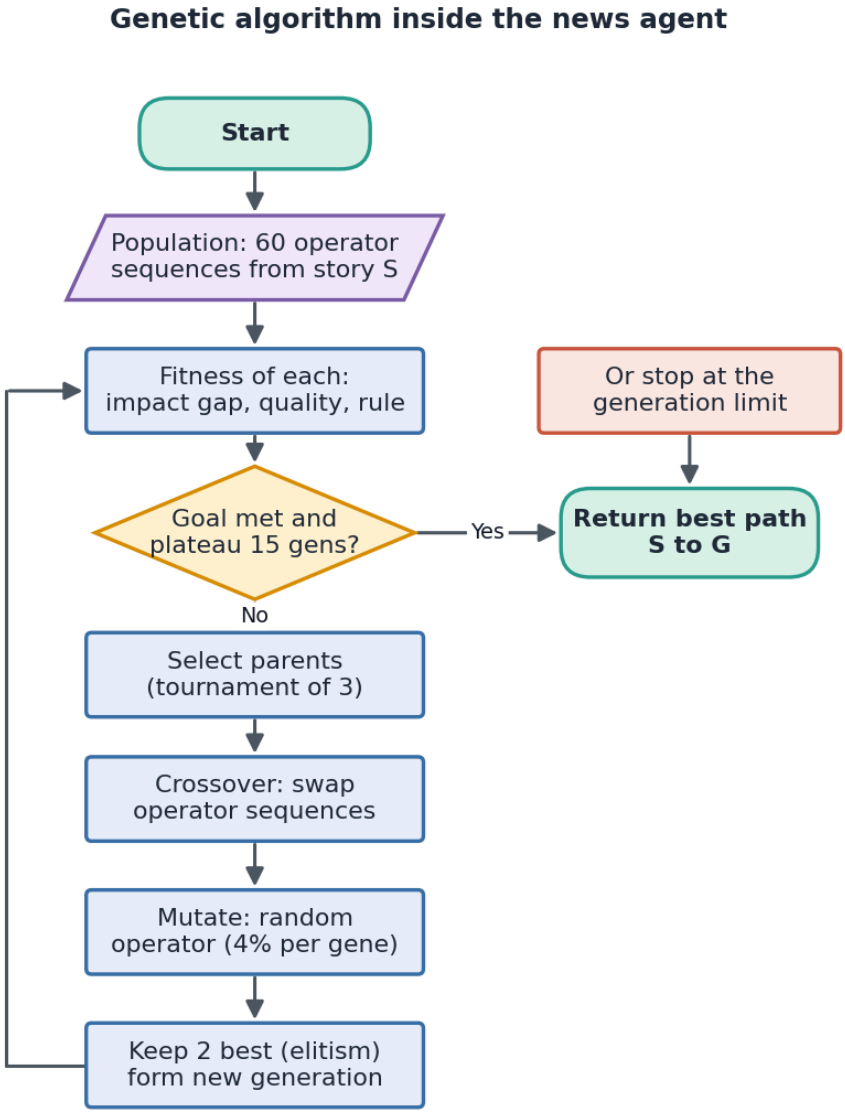



Figure 1. Flowchart of the GA inside the news agent.

## 5. Pseudocode

```
function NewsAgent(S, G):                                    # goal-based agent function
    population ← 60 random operator sequences (length 30)
    for generation = 0 .. MAX_GEN:
        for each seq in population:
            path ← apply operators from S, skip moves that leave the map
            fitness(seq) ← closeness + reached + quality + checkpoints
                           − penalty(black squares in a row > 2) − length cost
        best ← sequence with the highest fitness
        if best reaches G and has no violations and no gain for 15 generations: return best
        new ← the 2 best sequences
        while new is not full:
            p1, p2 ← tournament_selection(population)
            child ← one_point_crossover(p1, p2)
            mutate each gene with probability 0.04
            new.add(child)
        population ← new
    return best
```

## 6. Exit criteria

A GA never knows it is "done", so we need clear stopping rules. The agent stops when **any** of these is true:

| Rule | Why |
|---|---|
| **Goal met:** best path ends on G with zero rule violations | The story reaches the target impact safely. This is the main rule. |
| **Plateau:** no improvement for 15 generations | More search is wasting time. Combined with the goal rule, it also lets the path get shorter first. |
| **Generation limit** (150) or time budget | The news deadline is fixed. The agent must publish something. |
| **Editor approval** (manual inspection) | A human can accept a good enough version at any time. |

The black-square rule is not an exit rule. It is a **feasibility check**: a path with violations can never be the final answer.

## 7. Result of a real run

| Generation | Best fitness | Average fitness |
|---:|---:|---:|
| 0 | 0.399 | −0.028 |
| 5 | 0.833 | 0.349 |
| 10 | 0.883 | 0.586 |
| 30 | 0.891 | 0.685 |
| **54 (exit)** | **0.904** | **0.805** |

The GA stopped at generation 54 (goal reached, no gain for 15 generations). The best path has **21 moves**, ends on G, visits only **2 black squares** (never in a row), has **0 violations** and passes both checkpoints C and D. The shortest possible walk is 17 moves, so the GA path is close to it.

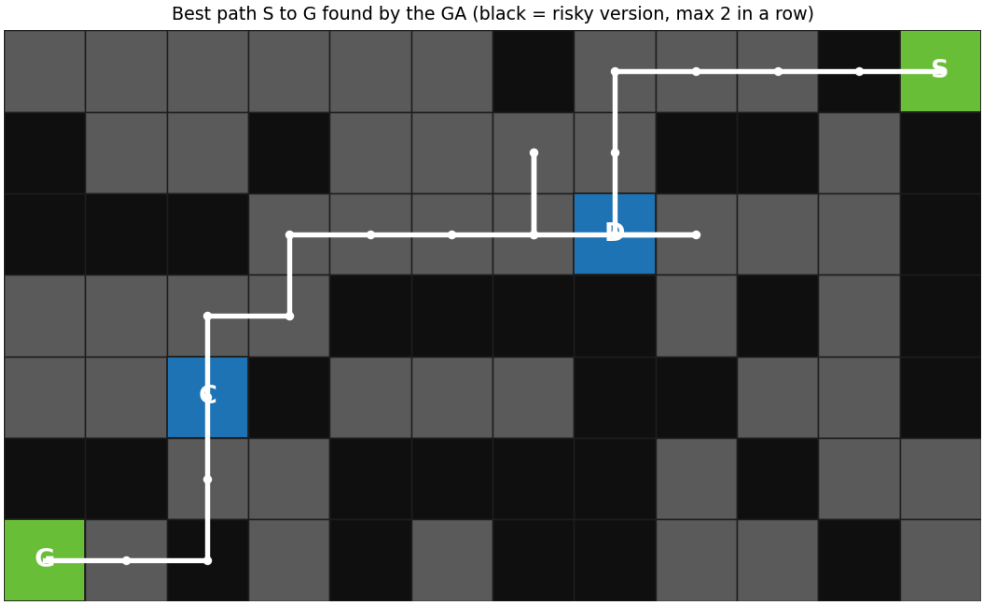



Figure 2. Diagram 2 recreated: the white line is the best path found by the GA. It passes D and C.

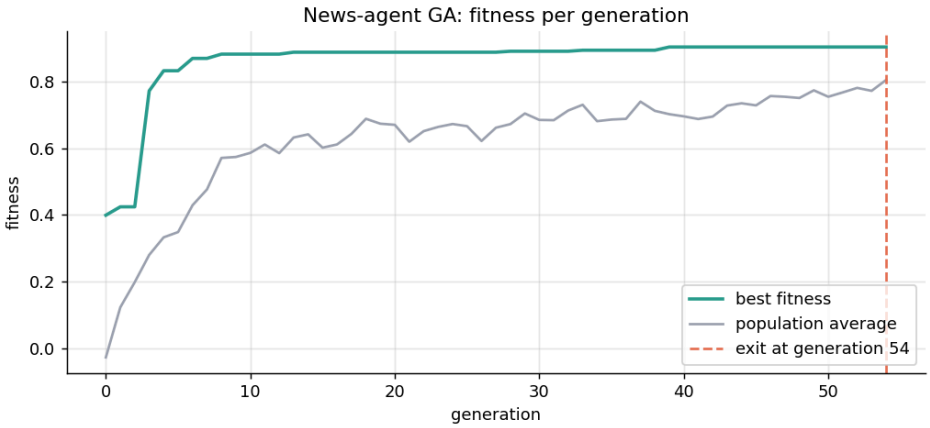



Figure 3. Fitness rises fast in the first 10 generations, then slowly improves until the exit.

## 8. Code and real run

File Edit View Run Terminal Help — q02_code.py - Q02_GA_News_Agent - Python IDE

q02_code.py

```python
def evolve():
    """GA: select -> crossover -> mutate, until the exit criteria are met."""
    pop = ["".join(rng.choice("LDRU") for _ in range(LENGTH)) for _ in range(POP)]
    best_hist, avg_hist, best, stale = [], [], pop[0], 0
    for gen in range(MAX_GEN):
        scores = [fitness(p) for p in pop]
        top = max(range(POP), key=lambda i: scores[i])
        best_hist.append(scores[top])
        avg_hist.append(sum(scores) / POP)
        stale = 0 if scores[top] > fitness(best) + 1e-9 else stale + 1
        best = pop[top] if scores[top] >= fitness(best) else best
        path, violations, _ = decode(best)
        if path[-1] == G and violations == 0 and stale >= PATIENCE:  # exit criterion
            return best, best_hist, avg_hist, gen, "goal reached, no gain for 15 generations"
        elite = sorted(range(POP), key=lambda i: -scores[i])[:2]
        nxt = [pop[i] for i in elite]
        while len(nxt) < POP:
            a, b = (max(rng.sample(range(POP), 3), key=lambda i: scores[i]) for _ in range(2))
            cut = rng.randrange(1, LENGTH)
            child = pop[a][:cut] + pop[b][cut:]  # one-point crossover
            nxt.append("".join(rng.choice("LDRU") if rng.random() < 0.04 else g for g in child))
        pop = nxt
    return best, best_hist, avg_hist, MAX_GEN, "generation limit"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Users\USER\questions\Answers\Q02_GA_News_Agent> python q02_code.py
Operators on the story 'SHOCK: city bridge collapses, thousands trapped!':
  L crossover          -> SHOCK: city bridge collapses | Council inspects Monday
  D insert fact        -> SHOCK: city bridge collapses, thousands trapped! Officials confirm 2 people hurt.
  R mutate tone        -> Bridge section collapses in city; rescue under way
  U delete sensational -> City bridge collapses, thousands trapped
Map: 7x12, 33 black squares. Population 60, genes 30.
 gen | best fitness | avg fitness
   0 |        0.399 | -0.028
   5 |        0.833 | 0.349
  10 |        0.883 | 0.586
  20 |        0.889 | 0.671
  30 |        0.891 | 0.685
  54 |        0.904 | 0.805
Exit at generation 54: goal reached, no gain for 15 generations
Best path: 21 moves, reaches G: True, black squares visited: 2, black-run violations: 0, checkpoints C/D visited: 2
Operators used: LUULLLUUUUDDRLLUDLLLD
charts saved to images/
PS C:\Users\USER\questions\Answers\Q02_GA_News_Agent>
```

main — Ln 81, Col 1 — Spaces: 4 — UTF-8 — CRLF — Python 3.12.3

Figure 4. The GA function (lines 59–81) and the real terminal output.

## 9. Summary

> The agent is goal-based, and its planning step is a GA. Each square is a news version, each move is one genetic operator, and fitness combines closeness to the target impact, quality, checkpoints and the black-square rule. The GA exited at generation 54 with a legal 21-move path from S to G. Clear exit rules (goal met, plateau, limit, editor) keep it from running forever.

---

Files in this folder: `q02_code.py` (source), `q02_code.ipynb` (Colab/Jupyter version, already run), `q02_output.txt` (console output), `images/` (all figures).